UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Pengelolaan Data Form Menggunakan PHP

Donny Reza, M.Kom

Mata Kuliah Aplikasi Teknologi Online

Program Studi Teknik Informatika, Fakultas Teknik dan Ilmu Komputer
Universitas Komputer Indonesia

Pengelolaan Data Form

Form HTML adalah elemen yang digunakan untuk mengumpulkan input dari pengguna dan mengirimkannya ke server untuk diproses.

PHP dapat menerima dan mengolah data yang dikirimkan melalui form menggunakan variabel superglobal $_POST dan $_GET.

Sebuah form HTML minimal terdiri dari:

- Tag <form> dengan atribut method dan action
- Elemen input seperti <input>, <textarea>, atau <select>
- Tombol submit untuk mengirimkan data

Pengelolaan Data Form

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Contoh HTML untuk menampilkan form

```
<form method="POST" action="proses.php">
    <input type="text" name="nama">
    <button type="submit">Kirim</button>
</form>
```



Atribut method menentukan cara pengiriman data (GET atau POST), sedangkan atribut action menentukan alamat file PHP yang akan memproses data tersebut.

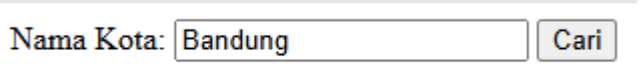
Metode GET

Metode GET mengirimkan data form melalui URL. Data yang dikirim akan terlihat langsung pada address bar browser dalam bentuk query string.

Contoh form dengan metode GET

form.html

```
<form method="GET" action="proses_get.php">
    <label>Nama Kota:</label>
    <input type="text" name="kota">
    <button type="submit">Cari</button>
</form>
```



proses_get.php

```
<?php

if (isset($_GET["kota"])) {
    $kota = $_GET["kota"];
    echo "Anda mencari kota: ".$kota;
} else {
    echo "Tidak ada data yang dikirimkan.";
}
```

Jika pengguna mengisi form dengan nilai “Bandung” lalu menekan tombol “Cari”, URL akan berubah menjadi: http://localhost/proses_get.php?kota=Bandung

Output PHP

```
Anda mencari kota: Bandung
```

Data yang dikirim melalui GET dapat dilihat oleh siapapun melalui URL, sehingga metode ini cocok digunakan untuk pencarian atau filter data, tetapi tidak disarankan untuk mengirim data sensitif seperti password.

Metode POST

Metode POST mengirimkan data form melalui body dari HTTP request, sehingga data tidak terlihat di URL. Metode ini lebih aman dibandingkan GET untuk mengirim data sensitif.

Contoh form dengan metode POST:

form.html

```
<!-- form.html -->
<form method="POST"
action="proses_post.php">
    <label>Nama:</label>
    <input type="text" name="nama">

    <label>Email:</label>
    <input type="email" name="email">

    <button type="submit">Kirim</button>
</form>
```

proses_get.php

```
<?php
// proses_post.php

if (isset($_POST["nama"]) &&
isset($_POST["email"])) {
    $nama  = $_POST["nama"];
    $email = $_POST["email"];

    echo "Nama  : ".$nama."<br>";
    echo "Email : ".$email;
} else {
    echo "Data tidak lengkap.";
}
```

Metode POST

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Contoh, jika form diisi sepert berikut

Nama: Budi Santoso Email: budi@email.com Kirim

Output PHP

```
Nama  : Budi Santoso
Email : budi@email.com
```

Perbandingan Metode GET dan POST

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Parameter	GET	POST
Data terlihat di URL	Ya	Tidak
Batas panjang data	Terbatas (~ 2000 karakter)	Tidak terbatas
Keamanan	Kurang aman	Lebih aman
Dapat di-bookmark	Ya	Tidak
Cocok digunakan untuk	Pencarian, filter, navigasi	Login, registrasi, pengiriman data sensitif
Dapat di-cache oleh browser	Ya	Tidak

Menangani Berbagai Jenis Input

Form HTML memiliki berbagai jenis elemen input. Berikut contoh form yang menggabungkan beberapa jenis input sekaligus beserta cara memprosesnya di PHP.

Contoh form Registrasi Sederhana

registrasi.html

```
    <p>
    <label>Kota Asal:</label>
    <select name="kota">
        <option value="Bandung">Bandung</option>
        <option value="Jakarta">Jakarta</option>
        <option value="Surabaya">Surabaya</option>
    </select>
    </p>
    <p>
    <label>Hobi:</label>
    <input type="checkbox" name="hobi[]" value="Membaca"> Membaca
    <input type="checkbox" name="hobi[]" value="Olahraga"> Olahraga
    <input type="checkbox" name="hobi[]" value="Memasak"> Memasak
    </p>
```

Menangani Berbagai Jenis Input

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

registrasi.html (lanjutan)

```
    <p>
    <label>Kota Asal:</label>
    <select name="kota">
        <option value="Bandung">Bandung</option>
        <option value="Jakarta">Jakarta</option>
        <option value="Surabaya">Surabaya</option>
    </select>
    </p>

    <p>
    <label>Hobi:</label>
    <input type="checkbox" name="hobi[]" value="Membaca"> Membaca
    <input type="checkbox" name="hobi[]" value="Olahraga"> Olahraga
    <input type="checkbox" name="hobi[]" value="Memasak"> Memasak
    </p>

    <p>
    <label>Tentang Saya:</label>
    <textarea name="tentang"></textarea>
    </p>
    <button type="submit">Daftar</button>
</form>
```

Menangani Berbagai Jenis Input

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Contoh tampilan form registrasi di browser

Nama Lengkap:

Jenis Kelamin: Laki-laki Perempuan

Kota Asal: Bandung

Hobi: Membaca Olahraga Memasak

Tentang Saya:

Daftar

Diisi

Nama Lengkap: Budi Santoso

Jenis Kelamin: Laki-laki Perempuan

Kota Asal: Bandung

Hobi: Membaca Olahraga Memasak

Tentang Saya: Saya seorang mahasiswa Teknik Informatika.

Daftar

Menangani Berbagai Jenis Input

Contoh file PHP untuk menangani data form

proses_registrasi.php

```
<?php

$nama           = $_POST["nama"];
$jenis_kelamin  = $_POST["jenis_kelamin"];
$kota           = $_POST["kota"];
$hobi           = $_POST["hobi"];
$tentang        = $_POST["tentang"];

echo "Nama           : ".$nama."<br>";
echo "Jenis Kelamin  : ".$jenis_kelamin."<br>";
echo "Kota Asal      : ".$kota."<br>";
echo "Hobi           : ".implode(", ", $hobi)."<br>";
echo "Tentang Saya   : ".$tentang;
```

Perhatikan beberapa hal pada contoh:

- Elemen <select> dikirim sebagai string biasa dan dapat diakses langsung menggunakan $_POST["kota"]
- Elemen <radio> hanya mengirimkan satu nilai sesuai pilihan pengguna
- Elemen <checkbox> dengan atribut name="hobi[]" menggunakan tanda [] agar PHP menerimanya sebagai array. Hasilnya diakses menggunakan $_POST["hobi"] yang bertipe array, dan dapat digabungkan menjadi string menggunakan fungsi implode()

Output PHP

```
Nama           : Budi Santoso
Jenis Kelamin  : Laki-laki
Kota Asal      : Bandung
Hobi           : Membaca, Olahraga
Tentang Saya   : Saya seorang mahasiswa Teknik Informatika.
```

Validasi Data Form

Data yang dikirim dari form harus selalu divalidasi sebelum diproses, untuk memastikan data yang diterima sesuai dengan yang diharapkan.

Contoh form nilai

form_nilai.html

```
<!-- form_nilai.html -->
<form method="POST" action="proses_nilai.php">
    <label>Nama Mahasiswa:</label>
    <input type="text" name="nama">

    <label>Nilai (0 - 100):</label>
    <input type="text" name="nilai">

    <button type="submit">Proses</button>
</form>
```

Validasi Data Form

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Contoh File PHP dengan proses validasi

proses_nilai.php

```
<?php
// proses_nilai.php

$error = [];

// Validasi nama
if (empty($_POST["nama"])) {
    $error[] = "Nama tidak boleh kosong.";
}

// Validasi nilai
if (!is_numeric($_POST["nilai"])) {
    $error[] = "Nilai harus berupa angka.";
} elseif ($_POST["nilai"] < 0 || $_POST["nilai"] > 100) {
    $error[] = "Nilai harus di antara 0 hingga 100.";
}
```

Validasi Data Form

proses_nilai.php (lanjutan)

```
// Menampilkan hasil
if (!empty($error)) {
    foreach ($error as $e) {
        echo "- ".$e."<br>";
    }
} else {
    $nama  = $_POST["nama"];
    $nilai = $_POST["nilai"];

    if ($nilai >= 80) {
        $grade = "A";
    } elseif ($nilai >= 70) {
        $grade = "B";
    } elseif ($nilai >= 60) {
        $grade = "C";
    } else {
        $grade = "D";
    }

    echo "Nama  : ".$nama."<br>";
    echo "Nilai : ".$nilai."<br>";
    echo "Grade : ".$grade;
}
```

Pada contoh, variabel $error berupa array yang menampung semua pesan kesalahan. Jika array $error tidak kosong, maka seluruh pesan kesalahan ditampilkan. Jika tidak ada kesalahan, data diproses dan hasilnya ditampilkan.

Output, jika nama = “Budi Santoso” dan nilai 85

```
Nama  : Budi Santoso
Nilai : 85
Grade : A
```

Output, jika nama tidak diisi dan nilai diisi “abc”

```
- Nama tidak boleh kosong.
- Nilai harus berupa angka.
```

Sanitasi Data Form

Selain validasi, data dari form juga perlu disanitasi untuk membersihkan karakter yang berpotensi berbahaya sebelum ditampilkan kembali ke halaman.

```php
<?php
// Contoh sanitasi data sebelum ditampilkan
$nama  = htmlspecialchars($_POST["nama"]);
$email = filter_var($_POST["email"], FILTER_SANITIZE_EMAIL);

echo "Nama  : ".$nama."<br>";
echo "Email : ".$email;
```

Fungsi htmlspecialchars() mengubah karakter khusus HTML menjadi *HTML entities* sehingga tidak dapat dieksekusi sebagai kode berbahaya di browser. Hal ini penting untuk mencegah serangan *Cross-Site Scripting* (XSS). Sedangkan filter_var() dengan parameter FILTER_SANITIZE_EMAIL akan menghapus karakter yang tidak valid dari alamat email.

Form dan Pengolahan Data Dalam Satu File

Pada contoh-contoh sebelumnya, form HTML dan script PHP dipisahkan ke dalam dua file yang berbeda. Namun, keduanya dapat digabungkan dalam satu file yang sama dengan memanfaatkan $_SERVER["PHP_SELF"] sebagai nilai atribut action, dan isset() untuk memeriksa apakah form sudah dikirimkan.

form_profil.php

```
<?php
// form_profil.php

$hasil = "";

if (isset($_POST["nama"])) {
    $nama   = htmlspecialchars($_POST["nama"]);
    $kota   = htmlspecialchars($_POST["kota"]);
    $hasil  = "Halo, ".$nama." dari ".$kota."!";
}
?>
```

Form dan Pengolahan Data Dalam Satu File

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

form_profil.php (lanjutan)

```
<form method="POST" action="<?php echo $_SERVER['PHP_SELF']; ?>">
    <label>Nama:</label>
    <input type="text" name="nama">

    <label>Kota:</label>
    <input type="text" name="kota">

    <button type="submit">Kirim</button>
</form>

<?php if ($hasil != "") { ?>
    <p><?php echo $hasil; ?></p>
<?php } ?>
```

Output, jika nama = “Budi Santoso” dan nilai “Bandung”

```
Halo, Budi Santoso dari Bandung!
```

Dengan pendekatan ini, form dan logika pemrosesan berada dalam satu file. Saat halaman pertama kali dibuka, isset($_POST["nama"]) akan bernilai false sehingga blok pemrosesan dilewati. Setelah form dikirimkan, isset($_POST["nama"]) menjadi true dan data akan diproses.